

EF Core 8.0 Hands-On Exercises

Lab 1: Understanding ORM with a Retail Inventory System

Scenario

You are building an inventory management system for a retail store. The store wants to track products, categories, and stock levels in a SQL Server database.

Objective

Understand what ORM is and how EF Core helps bridge the gap between C# objects and relational tables.

Step 1: What is ORM?

ORM stands for Object Relational Mapper. It acts as a translator between your C# code and a relational database, allowing developers to work with database records as C# objects instead of writing raw SQL queries.

How ORM Maps C# Classes to Database Tables

C# World	Database World	Example
Class	Table	Product class → Products table
Property	Column	Name property → Name column
Object instance	Row / Record	new Product() → one row
List of objects	Multiple rows	List<Product> → all rows
int Id (Primary Key)	Primary Key Column	Id → INT PRIMARY KEY

Without ORM (Raw SQL)

```
// Developer must write SQL manually
string sql = "SELECT * FROM Products WHERE Price > 50000";
SqlCommand cmd = new SqlCommand(sql, connection);
SqlDataReader reader = cmd.ExecuteReader();
// Manual mapping needed for every field...
```

With ORM (EF Core)

```
// Clean C# code - no SQL needed!
var products = await context.Products
    .Where(p => p.Price > 50000)
    .ToListAsync();
```

Benefits of ORM

- Productivity - Write C# instead of SQL, faster development
- Maintainability - Change C# class = automatically change DB schema
- Abstraction - No need to know SQL deeply to work with databases
- Type Safety - Compile-time checking of queries and models
- Cross-database - Switch between SQL Server, PostgreSQL, SQLite easily

Step 2: EF Core vs EF Framework (EF6)

Feature	EF Core 8.0	EF Framework (EF6)
Platform	Cross-platform (Windows, Mac, Linux)	Windows only
Performance	Faster and lightweight	Slower, heavier
LINQ Support	Full modern support	Partial support
Async Queries	Full async/await support	Limited

Feature	EF Core 8.0	EF Framework (EF6)
Compiled Queries	Yes - optimized performance	No
JSON Columns	Yes (EF Core 7+)	No
Bulk Operations	ExecuteUpdateAsync / ExecuteDeleteAsync	Not supported natively
.NET Version	.NET 6, 7, 8 (modern)	.NET Framework only
Open Source	Yes - fully open source	Partially open source
Best For	All new projects	Old legacy applications only

Conclusion: Always use EF Core 8.0 for new projects. EF6 is only for maintaining older .NET Framework applications.

Step 3: EF Core 8.0 New Features

1. JSON Column Mapping

Store complex nested objects directly as JSON in a single database column, allowing rich data structures without extra tables.

```
public class Product {
    public int Id { get; set; }
    public string Name { get; set; }
    public ProductDetails Details { get; set; } // Saved as JSON!
}
```

```
public class ProductDetails {
    public string Color { get; set; }
    public string Size { get; set; }
    public string Warranty { get; set; }
}
```

```
}
```

2. Compiled Models - Improved Performance

Pre-compile your EF Core model at build time for much faster application startup. Especially useful for large models with many entities.

```
// Run this CLI command to generate compiled model
dotnet ef dbcontext optimize

// Then use in your DbContext
optionsBuilder.UseModel(CompiledModels.AppDbContextModel.Instance);
```

3. Interceptors and Better Bulk Operations

EF Core 8.0 allows direct database updates and deletes without loading data into memory, making bulk operations extremely fast.

```
// OLD WAY - loads all into memory (slow for 1000+ records)
var products = await context.Products.ToListAsync();
foreach (var p in products) p.Price *= 0.9m;
await context.SaveChangesAsync();

// NEW EF CORE 8 WAY - single SQL statement (fast!)
await context.Products
    .Where(p => p.CategoryId == 1)
    .ExecuteUpdateAsync(p => p.SetProperty(x => x.Price, x => x.Price *
0.9m));

// Bulk delete without loading into memory
await context.Products
    .Where(p => p.StockQuantity == 0)
    .ExecuteDeleteAsync();
```

Step 4: Create a .NET Console App

```
1.dotnet new console -n RetailInventory

2.cd RetailInventory

3.Verify the project was created
```

- PS E:\Digital_Nurture_DotNet> **cd** RetailInventory
- PS E:\Digital_Nurture_DotNet\RetailInventory> **dir**

Directory: E:\Digital_Nurture_DotNet\RetailInventory

Mode	LastWriteTime	Length	Name
d----	24-06-2026 12:51		obj
-a----	24-06-2026 12:51	105	Program.cs
-a----	24-06-2026 12:51	252	RetailInventory.csproj

Step 5: Install EF Core Packages

```
# Install SQL Server provider for EF Core
dotnet add package Microsoft.EntityFrameworkCore.SqlServer

# Install Design package (required for migrations CLI)
dotnet add package Microsoft.EntityFrameworkCore.Design
```

```
PS E:\Digital_Nurture_DotNet\RetailInventory> dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 8.0.13
content hash BiIg+KwaSDOITze6jGQynxg64naAPtqGHBwDrLaCtixsa5bKiR8dpPOHA7ge3C0JJQizJE+sfkz1wV+BAKAYZw==.
info : Installed System.Runtime 4.3.0 from https://api.nuget.org/v3/index.json to E:\NugetCache\packages\system.runtime\4.3.0 with content hash JufQioVPQ0xGnAczR13AUFglDyVYt4Kqnz1AZaKZ5+GICq0/1MH/mO/eAJHT/mHw1zjKBjd7kV26SrxddAhiw==.
info : CACHE https://api.nuget.org/v3/vulnerabilities/index.json
info : CACHE https://api.nuget.org/v3-vulnerabilities/2026.06.19.23.48.43/vulnerability.base.json
info : CACHE https://api.nuget.org/v3-vulnerabilities/2026.06.19.23.48.43/2026.06.24.05.48.59/vulnerability.update.json
info : Package 'Microsoft.EntityFrameworkCore.SqlServer' is compatible with all the specified frameworks in project 'E:\Digital_Nurture_DotNet\RetailInventory\RetailInventory.csproj'.
info : PackageReference for package 'Microsoft.EntityFrameworkCore.SqlServer' version '8.0.13' added to file 'E:\Digital_Nurture_DotNet\RetailInventory\RetailInventory.csproj'.
info : Generating MSBuild file E:\Digital_Nurture_DotNet\RetailInventory\obj\RetailInventory.csproj.nuget.g.props.
info : Generating MSBuild file E:\Digital_Nurture_DotNet\RetailInventory\obj\RetailInventory.csproj.nuget.g.targets.
info : Writing assets file to disk. Path: E:\Digital_Nurture_DotNet\RetailInventory\obj\project.assets.json
log : Restored E:\Digital_Nurture_DotNet\RetailInventory\RetailInventory.csproj (in 18.56 sec).
```

```
PS E:\Digital_Nurture_DotNet\RetailInventory> dotnet add package Microsoft.EntityFrameworkCore.Design --version 8.0.13
info : Installed Microsoft.CodeAnalysis.CSharp 4.5.0 from https://api.nuget.org/v3/index.json to E:\NugetCache\packages\microsoft.codeanalysis.csharp\4.5.0 with content hash cM59wMKA0xvdv76bdmAKPy5hfj+oR+zxikWoueEB7CwTko7mt9sVKZI8Qxlov0C/LuKEG+uQwifepqL3vuTiBQ==.
info : Installed Microsoft.CodeAnalysis.Analyzers 3.3.3 from https://api.nuget.org/v3/index.json to E:\NugetCache\packages\microsoft.codeanalysis.analyzers\3.3.3 with content hash j/roZtLMVJjrFLRLAMckJLPW/1rze9MT1yfwqSIbUPGRu1m1P0fu09PmqapwsmePFGB5PJrudQLvmUOAMF0DqQ==.
info : CACHE https://api.nuget.org/v3/vulnerabilities/index.json
info : CACHE https://api.nuget.org/v3-vulnerabilities/2026.06.19.23.48.43/vulnerability.base.json
info : CACHE https://api.nuget.org/v3-vulnerabilities/2026.06.19.23.48.43/2026.06.24.05.48.59/vulnerability.update.json
info : Package 'Microsoft.EntityFrameworkCore.Design' is compatible with all the specified frameworks in project 'E:\Digital_Nurture_DotNet\RetailInventory\RetailInventory.csproj'.
info : PackageReference for package 'Microsoft.EntityFrameworkCore.Design' version '8.0.13' added to file 'E:\Digital_Nurture_DotNet\RetailInventory\RetailInventory.csproj'.
info : Generating MSBuild file E:\Digital_Nurture_DotNet\RetailInventory\obj\RetailInventory.csproj.nuget.g.props.
info : Writing assets file to disk. Path: E:\Digital_Nurture_DotNet\RetailInventory\obj\project.assets.json
log : Restored E:\Digital_Nurture_DotNet\RetailInventory\RetailInventory.csproj (in 7.61 sec).
```

What Each Package Does

Package	Purpose
EntityFrameworkCore.SqlServer	Connects EF Core to SQL Server database
EntityFrameworkCore.Design	Enables CLI tools like migrations and scaffolding